Week 4 Assignment Report

🎯 **Objective**

The objective of this assignment is to develop practical skills in Statistics, Data Visualization, and Exploratory Data Analysis (EDA). Students will apply statistical concepts, create visualizations using Matplotlib and Seaborn, clean and analyze datasets, identify correlations and outliers, and generate meaningful insights and business recommendations.

Task 1: Statistical Analysis – Mean, Median & Mode
Using a suitable numerical dataset:

Calculate the Mean. Calculate the Median. Calculate the Mode. Display the results. Write a brief explanation of what each measure tells you about the dataset.

----------**code**----------

```
import seaborn as sns

# Load the Titanic dataset using Seaborn

df = sns.load_dataset("titanic")

# Select the Age column and remove missing values

age = df["age"].dropna()

# Calculate Mean

mean_age = age.mean()

# Calculate Median

median_age = age.median()

# Calculate Mode

mode_age = age.mode()[0]

# Display the results

print("Mean:", mean_age)
```

```
print("Median:", median_age)

print("Mode:", mode_age)
```

----------output----------

```
Mean: 29.69911764705882
Median: 28.0
Mode: 24.0
```

Explanation

- **Mean: The average age of passengers is approximately 29.70 years.**
- **Median: The middle age of passengers is 28 years.**
- **Mode: The most frequently occurring age is 24 years.**

These measures describe the central tendency of the passenger age data.

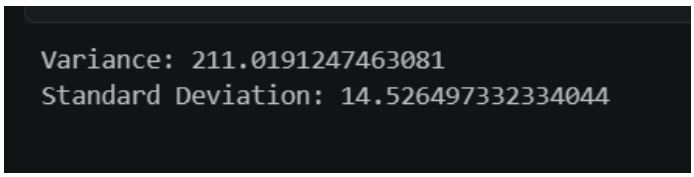
Task 2: Variance & Standard Deviation

Using the same or another suitable dataset: Calculate Variance. Calculate Standard Deviation. Display the results. Explain what the standard deviation indicates about the spread of the data.

----------**code**----------

```
import seaborn as sns

# Load the Titanic dataset using Seaborn

df = sns.load_dataset("titanic")

# Select the Age column and remove missing values

age = df["age"].dropna()

# Calculate Variance

variance_age = age.var()

# Calculate Standard Deviation

std_age = age.std()

# Display the results

print("Variance:", variance_age)

print("Standard Deviation:", std_age)
```

----------**output**----------



Explanation

- **Variance:** Variance measures how much the ages differ from the mean age.
- **Standard Deviation:** The standard deviation is approximately **14.53 years**.
- This means that passenger ages typically vary by about **14.53 years from the mean age**. A higher standard deviation indicates greater spread in the data, while a lower standard deviation indicates that values are closer to the mean.

Task 3: Correlation & Probability Basics

Perform the following: Correlation Select two numerical variables from a dataset. Calculate their correlation. Determine whether the relationship is positive, negative, or weak. Probability Create a simple real-world probability example. Calculate the probability. Display and explain the result.

----------**code**----------

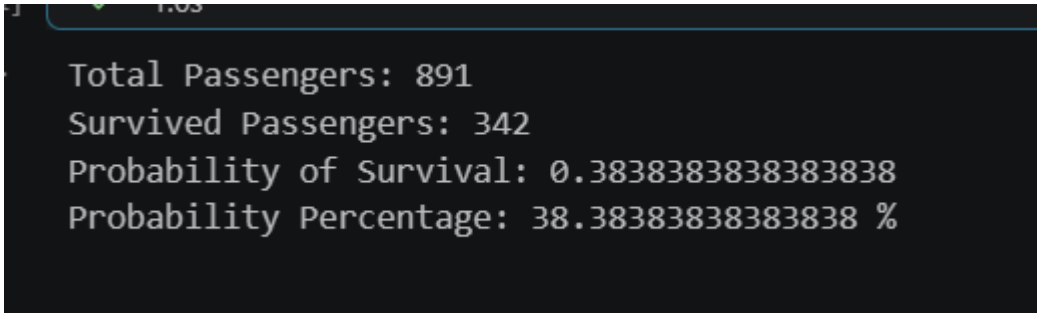
```
import seaborn as sns

# Load the Titanic dataset

df = sns.load_dataset("titanic")

# Select Age and Fare and remove missing values

correlation_data = df[["age", "fare"]].dropna()

# Calculate correlation

correlation = correlation_data["age"].corr(correlation_data["fare"])

# Display the correlation

print("Correlation between Age and Fare:", correlation)

# Determine the type of relationship

if correlation > 0.5:

    print("Strong Positive Relationship")

elif correlation > 0:

    print("Weak Positive Relationship")

elif correlation < -0.5:

    print("Strong Negative Relationship")

elif correlation < 0:

    print("Weak Negative Relationship")

else:

    print("No Significant Relationship")
```

```
Correlation between Age and Fare: 0.09606669176903888
Weak Positive Relationship
```

Explanation

The correlation between age and fare is **weakly positive**. This means that older passengers tend to have slightly higher fares, but the relationship is not strong.

```
import seaborn as sns

# Load the Titanic dataset

df = sns.load_dataset("titanic")

# Count total passengers

total_passengers = len(df)

# Count passengers who survived

survived_passengers = df["sur survived"].sum() if "sur survived" in df.columns else df["survived"].sum()

# Calculate probability

probability = survived_passengers / total_passengers

# Display the result

print("Total Passengers:", total_passengers)

print("Survived Passengers:", survived_passengers)

print("Probability of Survival:", probability)

print("Probability Percentage:", probability * 100, "%")
```



Explanation

The probability of randomly selecting a passenger who survived is approximately **0.384**, or **38.38%**.

Task 4: Outlier Detection

Using a numerical dataset: Identify potential outliers. Use an appropriate method to detect the outliers. Display the identified outliers. Briefly explain how outliers can affect data analysis.

----------**code**----------

```
import seaborn as sns

# Load the Titanic dataset

df = sns.load_dataset("titanic")

# Select Fare column and remove missing values

fare = df["fare"].dropna()

# Calculate Q1 and Q3

Q1 = fare.quantile(0.25)

Q3 = fare.quantile(0.75)

# Calculate IQR

IQR = Q3 - Q1

# Calculate lower and upper limits

lower_limit = Q1 - 1.5 * IQR

upper_limit = Q3 + 1.5 * IQR

# Identify outliers

outliers = fare[(fare < lower_limit) | (fare > upper_limit)]

# Display results

print("Q1:", Q1)

print("Q3:", Q3)

print("IQR:", IQR)

print("Lower Limit:", lower_limit)

print("Upper Limit:", upper_limit)

print("\nNumber of Outliers:", len(outliers))

print("\nOutliers:")
```

print(outliers)

----------output----------

```
Q1: 7.9104
Q3: 31.0
IQR: 23.0896
Lower Limit: -26.724
Upper Limit: 65.6344

Number of Outliers: 116

Outliers:
1       71.2833
27     263.0000
31     146.5208
34      82.1708
52      76.7292
         ...
846     69.5500
849     89.1042
856    164.8667
863     69.5500
879     83.1583
Name: fare, Length: 116, dtype: float64
```

Explanation

The **IQR method** identifies values that are unusually far from the middle 50% of the data.

The Titanic dataset contains several unusually high fare values. These are potential **outliers**.

Effect of outliers

Outliers can:

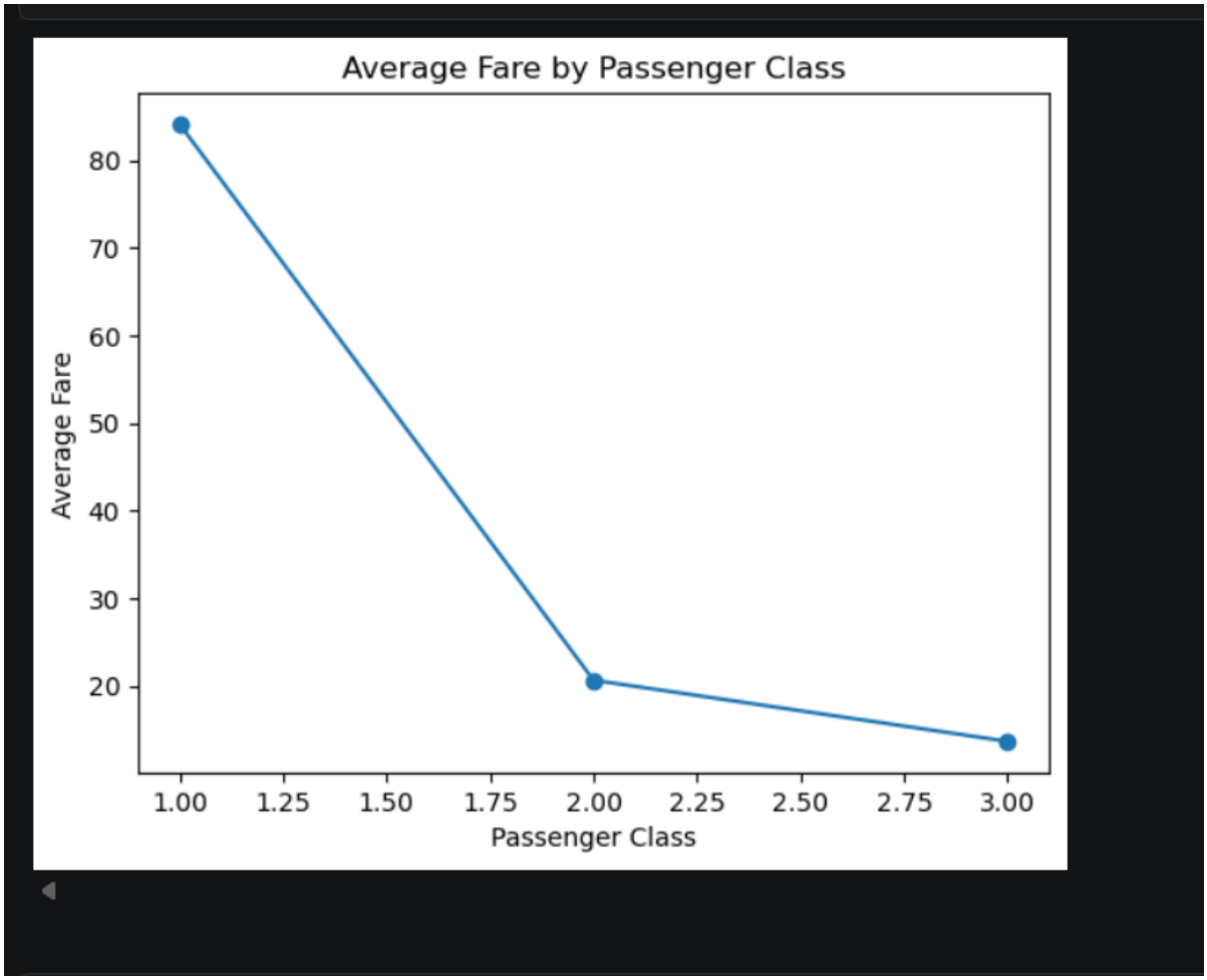
- Increase the mean.
- Increase variance and standard deviation.
- Affect correlation results.
- Influence statistical analysis and machine-learning models.

Task 5: Matplotlib Visualization

Using a suitable dataset, create the following visualizations using Matplotlib: Line Chart Bar Chart Pie Chart Histogram Scatter Plot For each visualization: Add an appropriate title. Label the axes where applicable. Use meaningful data. Write a brief explanation of the visualization.

----------**code**----------

```
import seaborn as sns

import matplotlib.pyplot as plt

# Load the Titanic dataset

df = sns.load_dataset("titanic")

# Calculate average fare for each passenger class

average_fare = df.groupby("pclass")["fare"].mean()

# Create Line Chart

plt.plot(average_fare.index, average_fare.values, marker="o")

# Add title and labels

plt.title("Average Fare by Passenger Class")

plt.xlabel("Passenger Class")

plt.ylabel("Average Fare")

# Display the chart

plt.show()
```

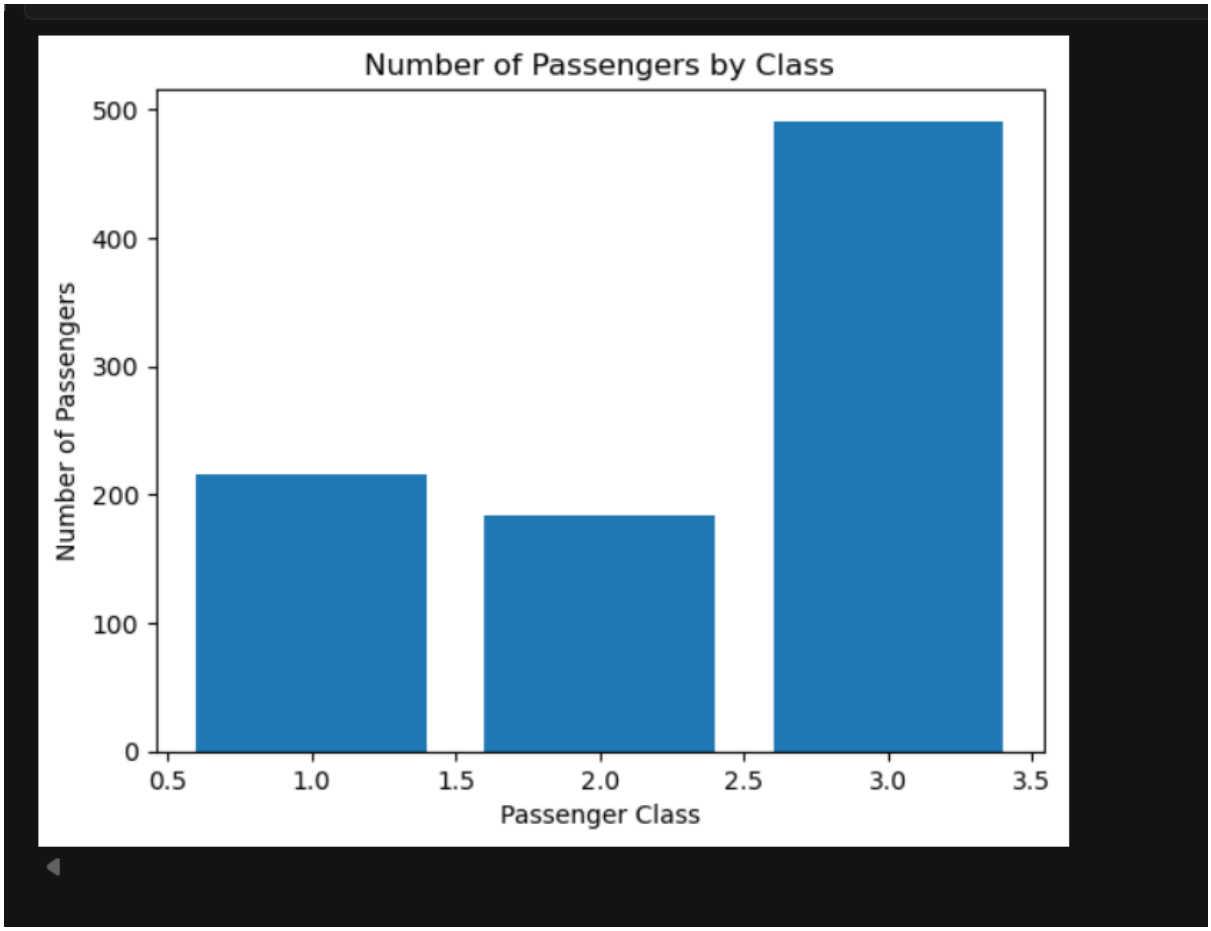


Explanation

The line chart shows how the average fare changes across passenger classes. **First-class passengers generally paid much higher fares than second- and third-class passengers.**

```
# Count passengers in each passenger class

class_count = df["pclass"].value_counts().sort_index()

# Create Bar Chart

plt.bar(class_count.index, class_count.values)

# Add title and labels

plt.title("Number of Passengers by Class")

plt.xlabel("Passenger Class")

plt.ylabel("Number of Passengers")
```

```
# Display the chart
plt.show()
```

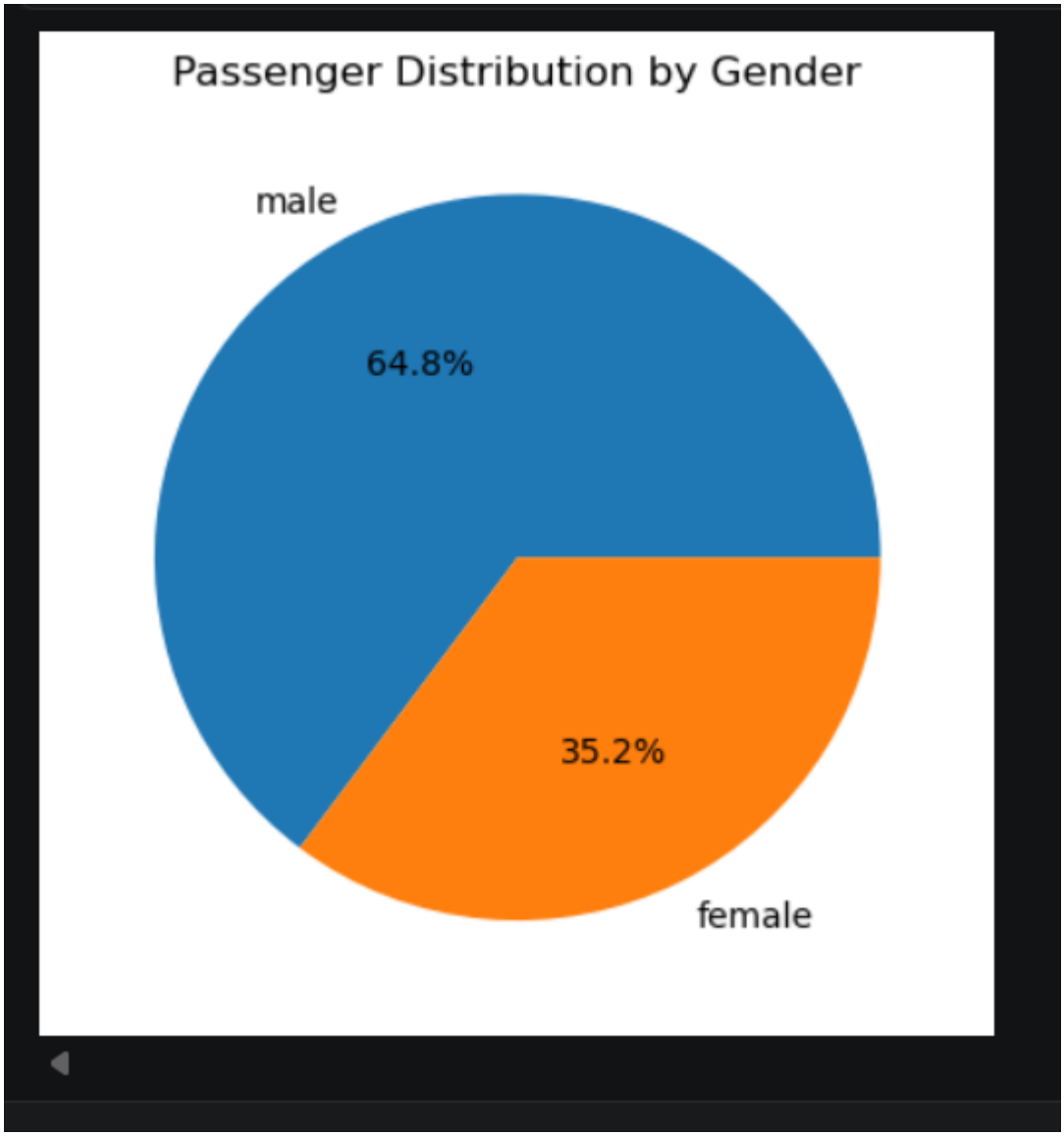


Explanation

The bar chart compares the number of passengers in each class. It clearly shows the distribution of passengers among first, second, and third class.

```
# Count male and female passengers
gender_count = df["sex"].value_counts()
# Create Pie Chart
plt.pie(gender_count.values, labels=gender_count.index, autopct="%1.1f%%")
# Add title
plt.title("Passenger Distribution by Gender")
```

```
# Display the chart
plt.show()
```

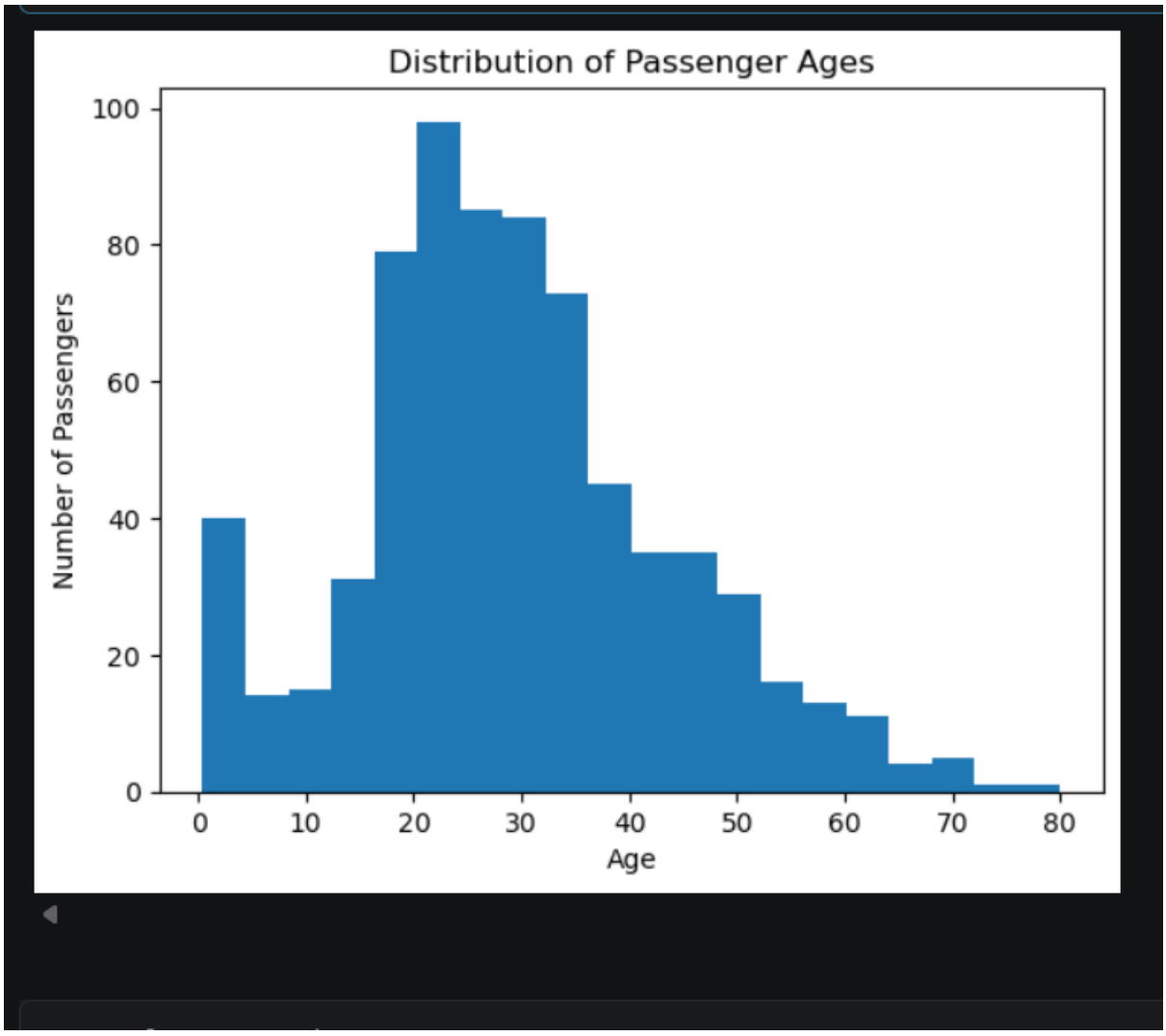


Explanation

The pie chart shows the proportion of male and female passengers in the Titanic dataset.

```
# Select Age column and remove missing values
age = df["age"].dropna()
# Create Histogram
plt.hist(age, bins=20)
```

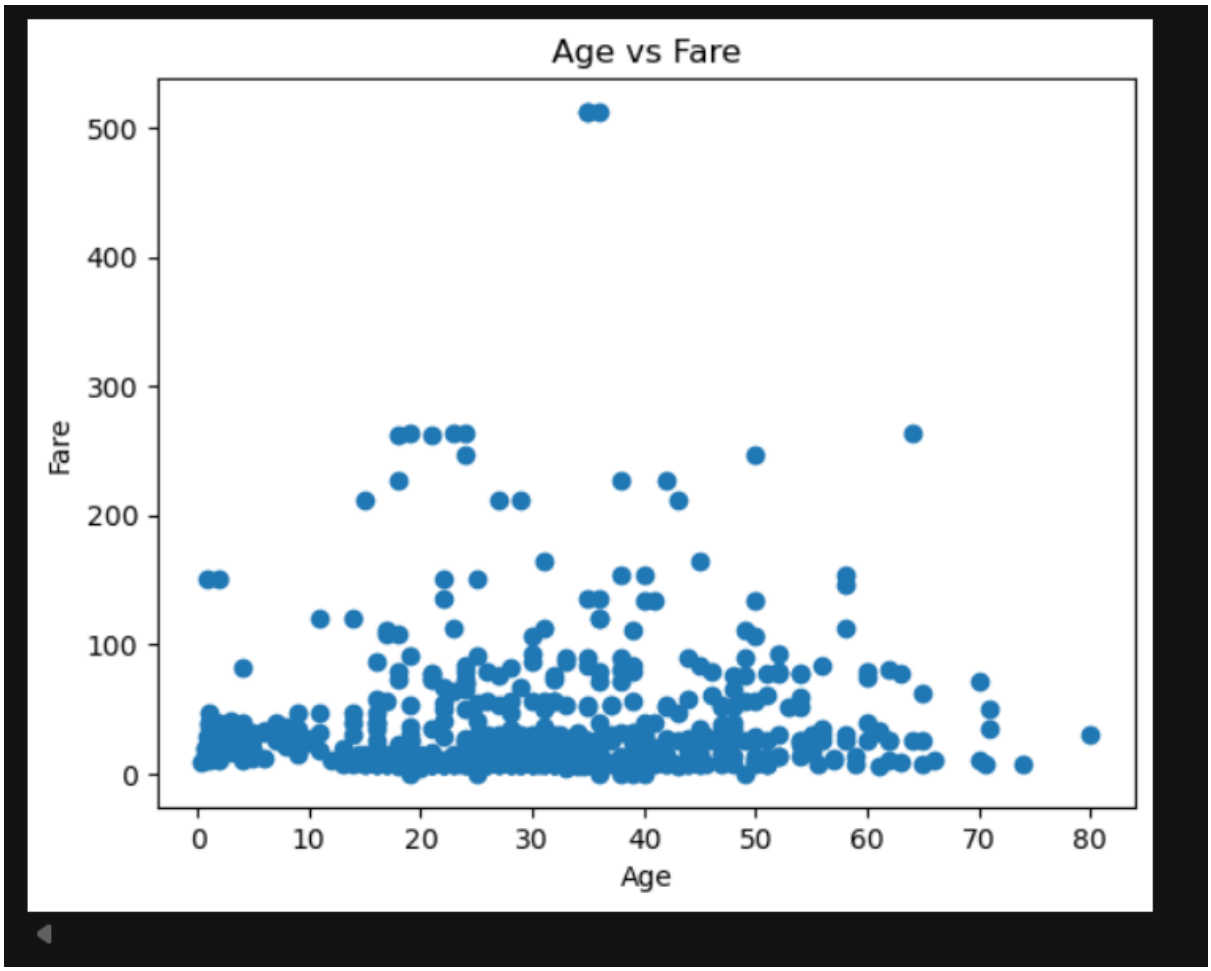
```
# Add title and labels
plt.title("Distribution of Passenger Ages")
plt.xlabel("Age")
plt.ylabel("Number of Passengers")
# Display the chart
plt.show()
```



Explanation

The histogram shows the distribution of passenger ages. Most passengers were concentrated in the younger and middle-age groups.

```
# Select Age and Fare
scatter_data = df[["age", "fare"]].dropna()
# Create Scatter Plot
plt.scatter(scatter_data["age"], scatter_data["fare"])
# Add title and labels
plt.title("Age vs Fare")
plt.xlabel("Age")
plt.ylabel("Fare")
# Display the chart
plt.show()
```



Explanation

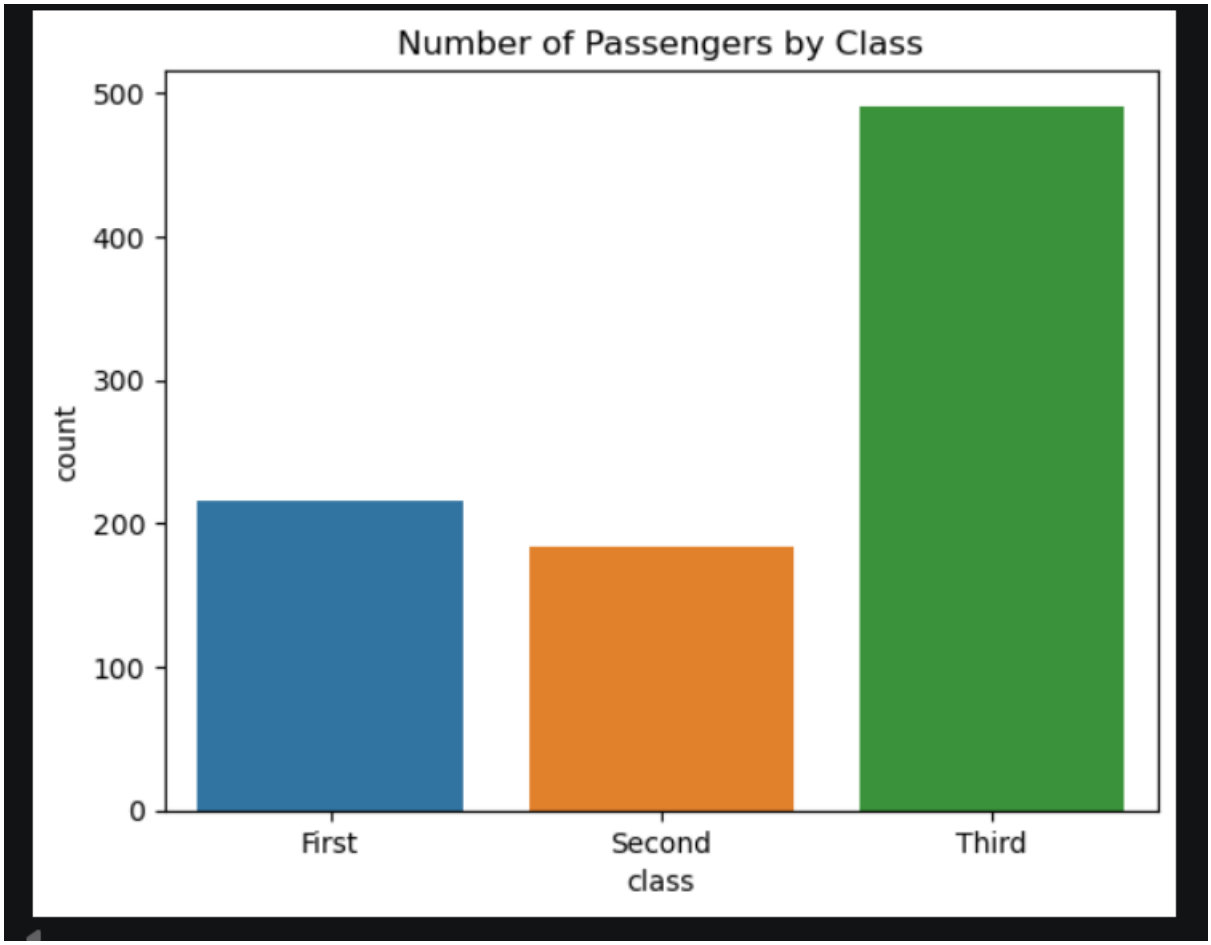
The scatter plot shows the relationship between age and fare. The points are widely scattered, indicating that there is **no strong relationship** between age and fare.

Task 6: Seaborn Visualization

Using a suitable dataset, create the following visualizations using Seaborn: Count Plot Box Plot Heatmap Pair Plot For each visualization: Add an appropriate title. Use relevant variables. Display the output. Write a brief explanation of the pattern or relationship shown**.**

----------**output**----------

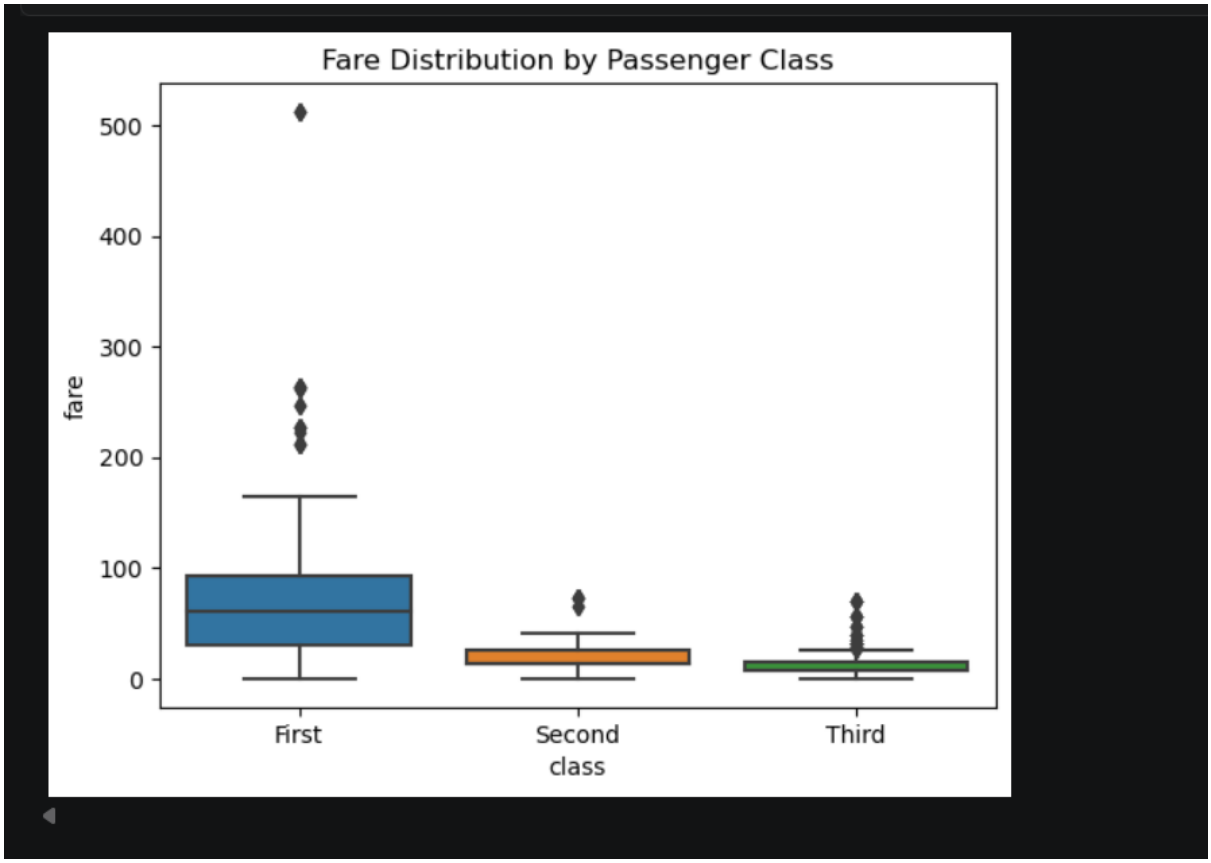
```
import seaborn as sns

import matplotlib.pyplot as plt

# Load the Titanic dataset

df = sns.load_dataset("titanic")

# Create Count Plot

sns.countplot(data=df, x="class")

# Add title

plt.title("Number of Passengers by Class")

# Display the chart

plt.show()
```



Explanation

The count plot shows the number of passengers in each passenger class. The **third class has the largest number of passengers**.

```
# Create Box Plot
sns.boxplot(data=df, x="class", y="fare")
# Add title
plt.title("Fare Distribution by Passenger Class")
# Display the chart
plt.show()
```

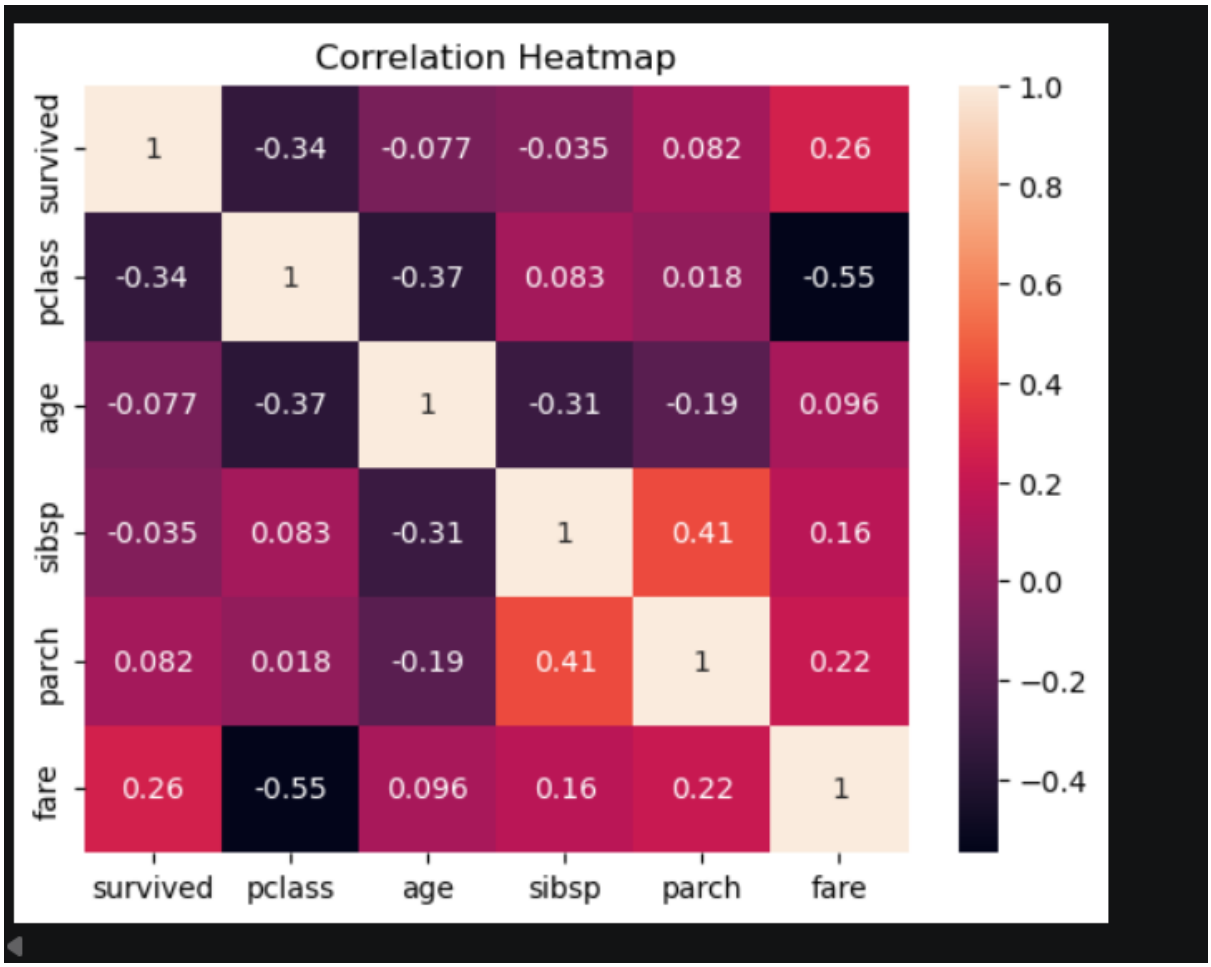


Explanation

The box plot compares fare distributions across passenger classes. First-class passengers have higher fares and also contain several high-value outliers.

```
# Select numerical columns

numeric_data = df.select_dtypes(include="number")

# Calculate correlation matrix

correlation_matrix = numeric_data.corr()

# Create Heatmap

sns.heatmap(correlation_matrix, annot=True)

# Add title

plt.title("Correlation Heatmap")

# Display the chart
```

plt.show()



Explanation

The heatmap displays the correlation between numerical variables. It helps identify variables that have positive or negative relationships with each other.

Select important numerical variables

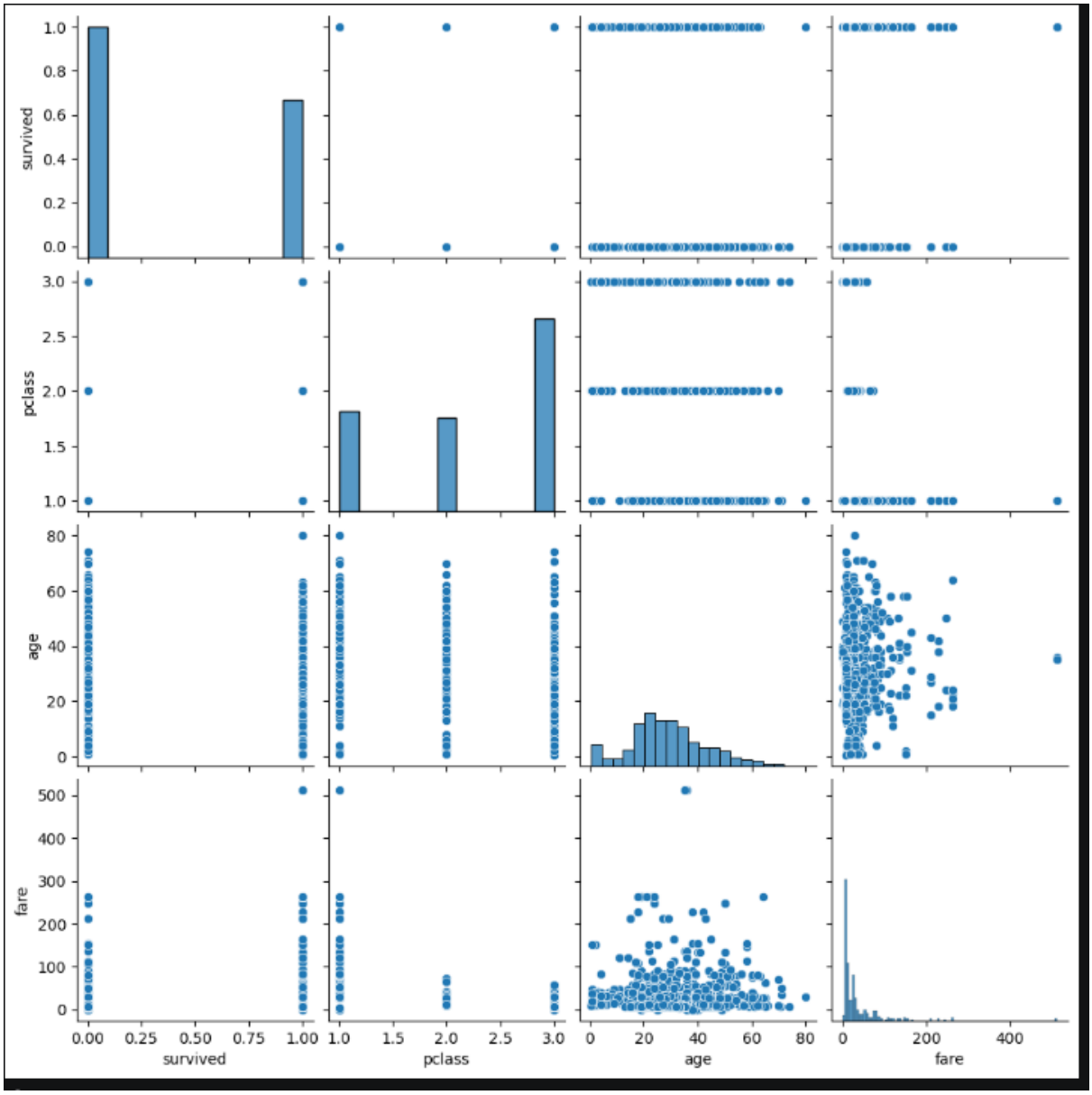
pair_data = df[["survived", "pclass", "age", "fare"]].dropna()

Create Pair Plot

sns.pairplot(pair_data)

Display the chart

plt.show()



Explanation

The pair plot shows relationships between multiple numerical variables at the same time. It helps identify patterns, trends, and possible relationships between variables.

Task 7: Exploratory Data Analysis (EDA) – Data Inspection & Cleaning

Choose a suitable dataset and perform: Data Inspection Check the number of rows and columns. Display column names. Check data types. Generate a statistical summary. Data Cleaning Identify missing values. Handle missing values appropriately. Check for duplicate records. Remove duplicates where required. Check the dataset for incorrect or inconsistent values. Display screenshots of the dataset before and after cleaning.

----------**output**----------

```
import seaborn as sns

# Load the Titanic dataset

df = sns.load_dataset("titanic")

# Display first 5 rows

print("First 5 Rows:")

print(df.head())

# Check number of rows and columns

print("\nNumber of Rows and Columns:")

print(df.shape)

# Display column names

print("\nColumn Names:")

print(df.columns)

# Check data types

print("\nData Types:")

print(df.dtypes)

# Display statistical summary

print("\nStatistical Summary:")

print(df.describe())

# Check missing values

print("\nMissing Values:")

print(df.isnull().sum())
```

```
# Create a copy of the dataset
df_clean = df.copy()
# Fill missing Age values with the median
df_clean["age"] = df_clean["age"].fillna(df_clean["age"].median())
# Fill missing Embarked values with the mode
df_clean["embarked"] = df_clean["embarked"].fillna(df_clean["embarked"].mode()[0])
# Fill missing Fare values with the median
df_clean["fare"] = df_clean["fare"].fillna(df_clean["fare"].median())
# Fill missing Deck values with Unknown
df_clean["deck"] = df_clean["deck"].astype("object").fillna("Unknown")
# Check duplicate records
print("\nNumber of Duplicate Records:")
print(df_clean.duplicated().sum())
# Remove duplicate records
df_clean = df_clean.drop_duplicates()
# Display missing values after cleaning
print("\nMissing Values After Cleaning:")
print(df_clean.isnull().sum())
# Display duplicate records after cleaning
print("\nDuplicate Records After Cleaning:")
print(df_clean.duplicated().sum())
# Display dataset shape after cleaning
print("\nDataset Shape After Cleaning:")
print(df_clean.shape)
# Display first 5 rows after cleaning
print("\nFirst 5 Rows After Cleaning:")
print(df_clean.head())
```

```
First 5 Rows:
   survived  pclass     sex   age  sibsp  parch     fare embarked  class  \
0         0       3    male  22.0      1      0   7.2500        S  Third
1         1       1  female  38.0      1      0  71.2833        C  First
2         1       3  female  26.0      0      0   7.9250        S  Third
3         1       1  female  35.0      1      0  53.1000        S  First
4         0       3    male  35.0      0      0   8.0500        S  Third

     who  adult_male deck  embark_town alive  alone
0    man        True  NaN  Southampton    no  False
1  woman       False    C    Cherbourg   yes  False
2  woman       False  NaN  Southampton   yes   True
3  woman       False    C  Southampton   yes  False
4    man        True  NaN  Southampton    no   True

Number of Rows and Columns:
(891, 15)

Column Names:
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
       'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
       'alive', 'alone'],
      dtype='object')

Data Types:
survived          int64
pclass            int64
sex              object
age             float64
sibsp             int64
```

```
sibsp             int64
parch             int64
fare            float64
embarked         object
class          category
who              object
adult_male         bool
deck           category
embark_town      object
alive            object
alone              bool
dtype: object

Statistical Summary:
        survived      pclass         age       sibsp       parch        fare
count  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean     0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std      0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min      0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%      0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%      0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%      1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max      1.000000    3.000000   80.000000    8.000000    6.000000  512.329200

Missing Values:
survived        0
pclass          0
sex             0
age           177
sibsp           0
```

```
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64

Number of Duplicate Records:
110

Missing Values After Cleaning:
survived       0
pclass         0
sex            0
age            0
sibsp          0
parch          0
fare           0
embarked       0
class          0
who            0
adult_male     0
deck           0
embark_town    2
```

```
alive          0
alone          0
dtype: int64

Duplicate Records After Cleaning:
0

Dataset Shape After Cleaning:
(781, 15)

First 5 Rows After Cleaning:
   survived  pclass     sex   age  sibsp  parch     fare embarked  class  \
0         0       3    male  22.0      1      0   7.2500        S  Third
1         1       1  female  38.0      1      0  71.2833        C  First
2         1       3  female  26.0      0      0   7.9250        S  Third
3         1       1  female  35.0      1      0  53.1000        S  First
4         0       3    male  35.0      0      0   8.0500        S  Third

     who  adult_male     deck  embark_town alive  alone
0    man        True  Unknown  Southampton    no  False
1  woman       False        C    Cherbourg   yes  False
2  woman       False  Unknown  Southampton   yes   True
3  woman       False        C  Southampton   yes  False
4    man        True  Unknown  Southampton    no   True
```

explanation

The Titanic dataset was inspected by checking its rows, columns, column names, data types, and statistical summary. Missing values were identified and handled using median and mode techniques. Duplicate records were checked and removed. Finally, the cleaned dataset was inspected again to verify that the data was properly cleaned.

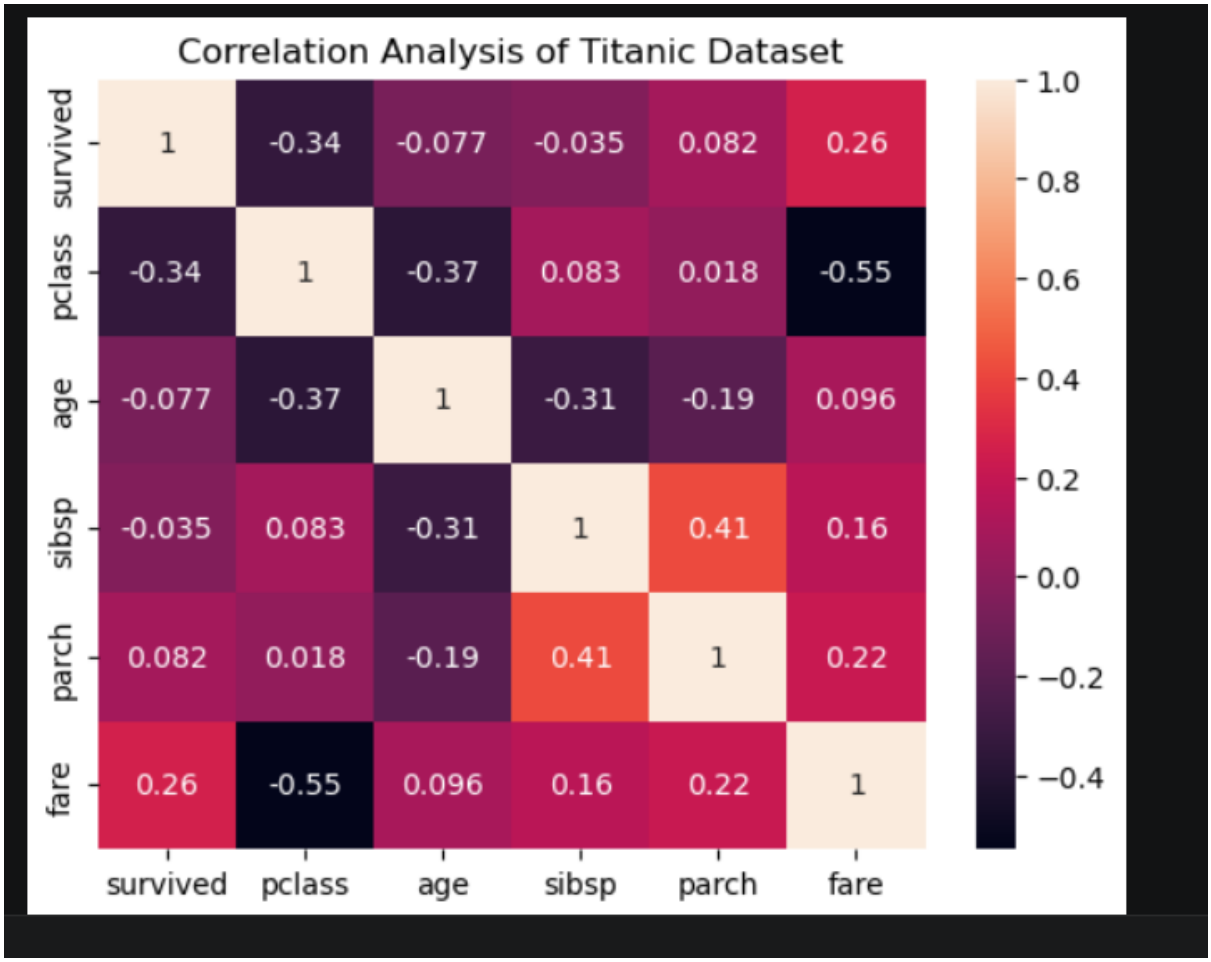
Task 8: EDA – Correlation & Insights

Continue the analysis of your selected dataset: Perform correlation analysis. Identify important relationships between variables. Identify patterns and trends. Use appropriate visualizations to support your findings. Write at least 3 meaningful insights from your analysis.

----------**output**----------

```
import seaborn as sns

import matplotlib.pyplot as plt

# Load the Titanic dataset

df = sns.load_dataset("titanic")

# Select numerical columns

numeric_data = df.select_dtypes(include="number")

# Calculate correlation matrix

correlation_matrix = numeric_data.corr()

# Display correlation matrix

print("Correlation Matrix:")

print(correlation_matrix)

# Create heatmap

sns.heatmap(correlation_matrix, annot=True)

# Add title

plt.title("Correlation Analysis of Titanic Dataset")

# Display the chart

plt.show()
```

```
Correlation Matrix:
          survived    pclass       age     sibsp     parch      fare
survived  1.000000 -0.338481 -0.077221 -0.035322  0.081629  0.257307
pclass   -0.338481  1.000000 -0.369226  0.083081  0.018443 -0.549500
age      -0.077221 -0.369226  1.000000 -0.308247 -0.189119  0.096067
sibsp    -0.035322  0.083081 -0.308247  1.000000  0.414838  0.159651
parch     0.081629  0.018443 -0.189119  0.414838  1.000000  0.216225
fare      0.257307 -0.549500  0.096067  0.159651  0.216225  1.000000
```



Insights

Insight 1:

There is a negative relationship between **passenger class and fare-related variables depending on coding**, while fare itself tends to be higher for passengers in better classes. The class variable is ordinal, so its correlation should be interpreted carefully.

Insight 2:

There is a relationship between **passenger class and survival**. Passengers from higher classes generally had better survival outcomes.

Insight 3:

Age and fare have a weak positive correlation, indicating that age alone does not strongly determine the fare paid by a passenger.

Task 9: Business Recommendations

Based on your EDA and findings: Provide at least 2 data-driven business recommendations. Explain how your recommendations are supported by the data.

Based on the analysis of the Titanic dataset, we can provide the following recommendations:

1. **Prioritize passenger safety:** Safety planning should consider passenger characteristics and location/class during emergencies.
2. **Improve emergency resource allocation:** Resources should be distributed efficiently across different passenger groups to improve survival outcomes.
3. **Use data-driven decision making:** Historical passenger data can be analyzed to identify patterns and improve future transportation safety policies.
4. **Monitor unusual values:** Outliers should be investigated before making important decisions because they can influence statistical analysis.

Conclusion

The analysis demonstrates how statistical measures, correlation, probability, outlier detection, visualization, and EDA can be used to understand a dataset and generate meaningful insights for decision-making.

❖Code link

https://drive.google.com/drive/folders/1suqCTtTR_OKbgtJqwRaV4bUaZd43DAXE?usp=sharing